

# Summer School on AI for Economics and Finance

## Deep Equilibrium Nets: from Brock–Mirman to loss balancing and architecture search

Simon Scheidegger

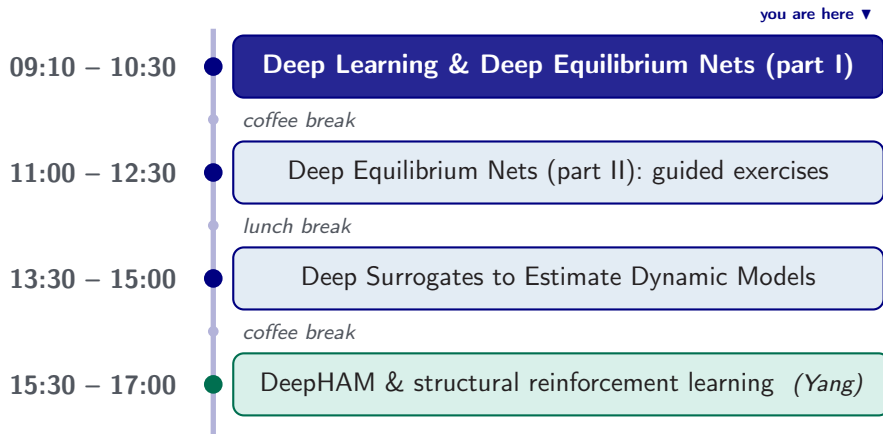
HEC, University of Lausanne

Based on Azinovic, Gaegauf & Scheidegger (2022), *International Economic Review* 63(4), 1471–1525

University of Torino

August 24, 2026

# Day 1 at a Glance



Materials: [github.com/sischei/summer-school-AI-...-2026](https://github.com/sischei/summer-school-AI-...-2026) · Cloud: [nuvolos.cloud](https://nuvolos.cloud)

# Roadmap of This Lecture (75–90 min)

## **I. Motivation** (12 min)

- ▶ Heterogeneity, high dimensions, kinks, irregular ergodic sets

## **II. From neural networks to DEQNs** (13 min)

- ▶ Self-supervised loss, the training algorithm, where DEQNs sit

## **III. Brock–Mirman benchmark** (25 min)

- ▶ Euler equation, hard constraints, residual and loss
- ▶ Quadrature, diagnostics, validation, loss kernels

## **IV. Loss balancing** (15 min)

- ▶ Pathology (a): components at different scales

## **V. Architecture search** (13 min)

- ▶ Pathology (b): random search and Hyperband

## **VI. Further pathologies, HPC, wrap-up** (12 min)

- ▶ Pathologies (c) to (e), data parallelism, hands-on preview

# Learning Objectives

By the end of this lecture you should be able to:

- ▶ Formulate dynamic stochastic economic models as equilibrium systems and identify the computational challenges in high dimensions
- ▶ Replace grid-based collocation with **self-supervised** neural-network training that uses equilibrium errors as the loss
- ▶ Build and train Deep Equilibrium Nets (DEQNs), and validate one against the Brock–Mirman closed form
- ▶ Diagnose and cure the two most common failure modes in practice: **loss-scale imbalance** and **poor architecture choice**
- ▶ Recognize the three residual-only pathologies that remain: lazy learning, learning from scratch, and self-confirming coverage
- ▶ Know when to reach for adaptive loss balancing (ReLoBRaLo), automated architecture search, and data parallelism

# Part I

## Motivation

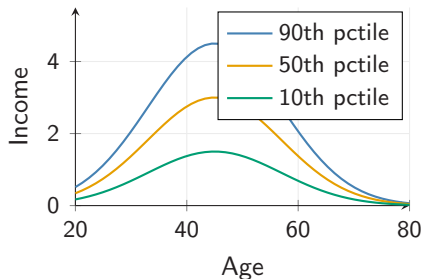
# Heterogeneity Matters

**The questions:** inequality over the life cycle, idiosyncratic versus aggregate risk, distributional effects of policy.

**The punchline:** **hand-to-mouth** households have a quarterly MPC of roughly 0.3 to 0.5, against about 0.05 for unconstrained ones. A representative agent averages these out and misses the policy response.

Kaplan, Moll & Violante 2018; Bilbiie 2008;  
Krueger, Mitman & Perri 2016

⇒ Heterogeneity is one of several forces that **blow up** the state dimension  $d_x$ .



Stylized income-by-age profiles

# Dynamic Stochastic General Equilibrium Models

**Generic formulation:** find policy functions  $p(\cdot)$  such that

$$\mathbb{E}_t[\mathcal{R}(\mathbf{x}_t, \mathbf{x}_{t+1}, p(\mathbf{x}_t), p(\mathbf{x}_{t+1}))] = 0$$

with transition law  $\mathbf{x}_{t+1} = h(\mathbf{x}_t, p(\mathbf{x}_t), \varepsilon_{t+1})$ , where  $\mathcal{R}$  collects the equilibrium conditions.

**Four computational challenges** (Azinovic, Gaegauf & Scheidegger, 2022):

1. **Stochasticity**: nontrivial expectations over  $\varepsilon_{t+1}$  and over the ergodic distribution
2. **High-dimensional** state space: heterogeneity, many assets, and many shocks all push  $d_x$  up
3. **Strong nonlinearities and kinks** from occasionally binding constraints and regime switches
4. **Irregular geometry** of the ergodic set: recurrent states are not a hypercube

No classical method addresses all four jointly: sparse grids cover (1)–(3) but become impractical past  $d_x \sim 20$ ; Gaussian processes cover (1), (2) and (4) but cost  $\mathcal{O}(n^3)$  to train.

# What Is High-Dimensional?

| Dimensions $d_x$ | Grid points $n_g^{d_x}$ | Runtime                 |
|------------------|-------------------------|-------------------------|
| 1                | 10                      | 10 seconds              |
| 2                | 100                     | 2 minutes               |
| 5                | $10^5$                  | 1 day                   |
| 10               | $10^{10}$               | 317 years               |
| 20               | $10^{20}$               | $\sim 3$ trillion years |

Assumes  $n_g = 10$  grid points per dimension and 1 second per evaluation. This is the curse of dimensionality of lecture 1, now with an economic state on the axis.

## Three remedies:

1. **Sparse grids** (Brumm & Scheidegger, 2017)
2. **Random grids** (Rust, 1997), for the restricted class of discrete decision processes
3. **Neural networks** (this lecture)

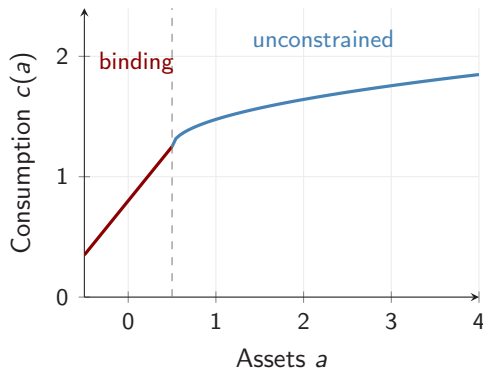
# Nonlinearities and Kinks

## Where do kinks come from?

- ▶ **Borrowing constraints:**  $a_{t+1} \geq \underline{a}$
- ▶ **Zero lower bound:**  $i_t \geq 0$
- ▶ **Short-sale and participation:** corner solutions
- ▶ **Default, regime switches, indivisibilities**

Occasionally binding constraints  $\Rightarrow$  policy functions are **non-smooth** at the boundary of the binding region.

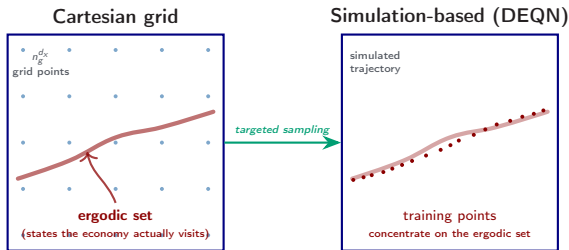
Polynomial and spectral methods lose accuracy; sparse grids can resolve kinks but only for  $d_x \lesssim 20$ . Neural networks handle kinks natively via ReLU-type activations.



Kinked consumption policy at the borrowing limit

# The Ergodic Set Is Not a Hypercube

**Grid-based** methods tile the full hypercube  $\mathbf{x} \in [\mathbf{x}_{\min}, \mathbf{x}_{\max}]^{d_x}$ . **But** simulated states drawn from the model's law of motion fill only a **thin, irregular** subset, the **ergodic set** of states the economy actually visits, often  $\ll 1\%$  of the hypercube (Judd, Maliar & Maliar, 2011).



A full Cartesian grid **wastes** most of its work in regions the economy never visits, which motivates **simulation-based, grid-free** methods.

# What We Need, and What DEQNs Deliver

**The wishlist.** A numerical method that is

1. **global**, not a local perturbation;
2. content with a small residual error;
3. **grid-free**, avoiding growth like  $n_g^{d_x}$ ;
4. comfortable with kinks and irregular ergodic sets;
5. **parallel-friendly**.

Three ingredients (Azinovic, Gaegauf & Scheidegger, 2022)

1. **Equilibrium error as the loss:** use  $\mathcal{R}(\cdot) = 0$  directly, with **no labeled data**
2. **Stochastic gradient descent** on the parameters  $\theta$
3. **Simulated paths:** draw training points from the model's own **ergodic distribution**

⇒ This turns **solving** an economic model into **training** a neural network, while **mitigating** rather than eliminating high-dimensional costs: expectation accuracy, sampling quality, and optimization all still matter.

# Part II

From Neural Networks to DEQNs

# What We Carry Over From Lecture 1

A deep network composes affine maps with activations, with  $\mathbf{a}^{(0)} = \mathbf{x}$ :

$$\mathbf{a}^{(\ell)} = g^{(\ell)}(\mathbf{W}^{(\ell)}\mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}), \quad \ell = 1, \dots, L-1, \quad \mathcal{N}_{\theta}(\mathbf{x}) = \mathbf{W}^{(L)}\mathbf{a}^{(L-1)} + \mathbf{b}^{(L)}.$$

Three facts we use:

- **Universal approximation**: for any continuous  $f$  on a compact set and any tolerance, *some* width suffices, though it may be large (Cybenko 1989; Hornik et al. 1989; Leshno et al. 1993).
- **Backprop is reverse-mode autodiff**: we get  $\nabla_{\theta}$  of *any* scalar built from  $\theta$ .
- **The output layer is a modeling choice**: softplus gives positivity, sigmoid a rate.

## Why this is the whole bridge

Lecture 1 ended by observing that autodiff differentiates any scalar built from  $\theta$ . So we may put an **Euler equation residual** in the loss and still get  $\nabla_{\theta}$  for free. That is exactly what DEQNs do.

**Activations.** ReLU handles **kinks**; tanh and Swish are **smooth**, which matters when derivatives of the network enter the loss.

# Notation for the Rest of the Lecture

| Symbol               | Meaning                          |
|----------------------|----------------------------------|
| $\mathbf{x}_t$       | state vector, dimension $d_x$    |
| $\varepsilon_{t+1}$  | shock, dimension $d_\varepsilon$ |
| $p(\cdot)$           | the true policy function         |
| $\mathcal{N}_\theta$ | its network approximation        |
| $\theta$             | network weights and biases       |
| $\mathcal{R}$        | equilibrium residual             |

| Symbol          | Meaning                           |
|-----------------|-----------------------------------|
| $J(\theta)$     | total training loss               |
| $\mathcal{L}_k$ | loss component, $k = 1, \dots, M$ |
| $\omega_k$      | weight on component $k$           |
| $N$             | number of sampled states          |
| $Q, w_q$        | quadrature nodes and weights      |
| $\eta$          | learning rate                     |

Economic parameters keep their usual names:  $\alpha$  capital share,  $\beta$  discount factor,  $\delta$  depreciation,  $\varrho$  persistence,  $\sigma_z$  shock volatility. Balancing and search hyperparameters carry subscripts ( $\alpha_{1b}$ ,  $\rho_{1b}$ ,  $\gamma$ ) so that they never collide with them.

# Supervised Learning and the “Data Problem” in Economics

The standard supervised recipe needs labeled data  $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$  and minimizes

$$J^{\text{sup}}(\theta) = \frac{1}{N} \sum_{i=1}^N \|y_i - \mathcal{N}_{\theta}(\mathbf{x}_i)\|^2,$$

by stochastic gradient descent, exactly as in lecture 1.

But in computational economics:

- ▶ We solve **functional equations**, not classification or regression tasks
- ▶ There are **no labeled pairs**  $(\mathbf{x}_i, y_i)$ : nobody hands us the true policy

## What we can exploit

We **know** the structural equations  $\mathcal{R}(\cdot) = 0$ , and we can **check** how well a candidate policy satisfies them.

⇒ Use the **equilibrium error** as the loss function.

# Traditional Approach: Time Iteration on a Grid

## Algorithm: grid-based collocation

```
Input: grid  $\mathcal{G} = \{\mathbf{x}_1, \dots, \mathbf{x}_{M_g}\} \subset \mathbb{R}^{d_x}$ , initial guess  $p^{(0)}$   
for  $k = 0, 1, 2, \dots$  until convergence do  
  for each grid point  $\mathbf{x}_i \in \mathcal{G}$  do  
    Solve  $\mathcal{R}(\mathbf{x}_i, p^{(k)}(\mathbf{x}_i), p^{(k)}) = 0$  for  $p^{(k+1)}(\mathbf{x}_i)$   
  end for  
  Interpolate  $p^{(k+1)}$  between grid points  
end for
```

## Problems:

- ▶ **Cartesian** grids need  $M_g = n_g^{d_x}$  points, infeasible beyond  $d_x \sim 4-5$  at sensible resolution ( $10^5$  at  $d_x=5$ ,  $10^{10}$  at  $d_x=10$ )
- ▶ **Sparse grids** (Smolyak, adaptive) extend the practical reach to roughly  $d_x \sim 20$
- ▶ Heterogeneous-agent and many-country models often need far more, which is beyond the reach of tensor grids

# The DEQN Loss Function

**The idea:** replace labeled data with **economic equilibrium conditions**. Let  $\mathcal{R}(\mathbf{x}, p(\mathbf{x})) = 0$  be the system of equilibrium equations, and define

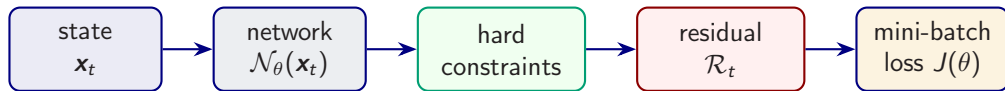
$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \|\mathcal{R}(\mathbf{x}_i, \mathcal{N}_{\theta}(\mathbf{x}_i))\|^2$$

- ▶  $\mathcal{N}_{\theta}(\mathbf{x}_i)$  is the network's **predicted policy** at state  $\mathbf{x}_i$ , and  $\mathcal{R}(\cdot)$  is the **equilibrium error**, for instance an Euler equation residual
- ▶ A small  $J(\theta)$  means the policy nearly satisfies the equations on the sampled states
- ▶ **No labels needed**. We call this **self-supervised** learning; it is sometimes labeled “unsupervised” in the narrow no-labels sense

## One caveat we will make precise in Part III

When  $\mathcal{R}$  contains a conditional expectation, it must be approximated *inside* the norm. Squaring a single simulated draw is a different, and biased, objective.

# Inside One DEQN Training Step



- ▶ **Hard constraints** are what the architecture can enforce exactly: budgets, positivity, market clearing.
- ▶ **Soft conditions** are what is left in the loss: Euler equations, KKT conditions, remaining equilibrium conditions.
- ▶ States are drawn from the model's own simulation, so training focuses on the ergodic set.

# Training Algorithm for DEQNs

## Algorithm 1: Deep Equilibrium Net training

**Input:** initial state  $\mathbf{x}_0$ , network  $\mathcal{N}_\theta$ , learning rate  $\eta$ , episodes  $E$ , horizon  $T_{\text{sim}}$ , steps per episode  $T_{\text{train}}$

**for** episode  $j = 1, \dots, E$  **do**

**Simulate path:**  $\mathbf{x}_0 \rightarrow \mathbf{x}_1 \rightarrow \dots \rightarrow \mathbf{x}_{T_{\text{sim}}}$  using  $\mathcal{N}_\theta$  and the transition law, then **detach** the states

**for** gradient step  $m = 1, \dots, T_{\text{train}}$  **do**

        Draw mini-batch  $\mathcal{B}$  of size  $B$  from  $\{\mathbf{x}_0, \dots, \mathbf{x}_{T_{\text{sim}}}\}$

$J(\theta) = \frac{1}{B} \sum_{\mathbf{x}_i \in \mathcal{B}} \|\mathcal{R}(\mathbf{x}_i, \mathcal{N}_\theta(\mathbf{x}_i))\|^2$

$\theta \leftarrow \theta - \eta \nabla_\theta J(\theta)$

**end for**

**Stop** when the test Euler-error quantiles stop improving over a window of episodes

**end for**

**Output:** trained network  $\mathcal{N}_{\theta^*}$  approximating the policy function

**Note.** Training points come from the **model's own simulation**, so the distribution the network is trained on *moves while training*. There is no contraction here and no convergence theorem: the stopping rule has to be empirical.

# Where DEQNs Sit in the Literature

## Classical global methods:

- ▶ Projection and collocation (Judd 1992)
- ▶ Parameterized expectations (den Haan & Marcet 1990)
- ▶ Krusell & Smith (1998)
- ▶ Adaptive sparse grids (Brumm & Scheidegger 2017)

## One sentence for macroeconomists

A DEQN is **parameterized expectations with a neural network**: the same regression on simulated data, with a flexible approximator and SGD in place of a linear projection.

## Deep-learning solvers:

- ▶ **DEQNs** (Azinovic, Gaegauf & Scheidegger 2022), this lecture
- ▶ Maliar, Maliar & Winant (2021), a unified framework
- ▶ Duarte (2018), continuous time
- ▶ Fernández-Villaverde et al. (2020), dynamic programming
- ▶ **DeepHAM** (Han, Yang & E 2026), *this afternoon at 15:30*
- ▶ Kahou et al. (2021), symmetry and exchangeability

# Part III

## Brock–Mirman Benchmark

# Brock–Mirman (1972): Setup

**Why this model.** It is the one growth model whose policy function is known in closed form, so every number a DEQN produces can be checked against the truth.

**Planner's problem.** Choose consumption to maximize expected lifetime log utility,

$$\max_{\{C_t\}_{t=0}^{\infty}} \mathbb{E} \left[ \sum_{t=0}^{\infty} \beta^t \ln C_t \right], \quad C_t + K_{t+1} = z_t K_t^\alpha, \quad K_0 > 0 \text{ given,}$$

with full depreciation  $\delta = 1$ , capital share  $\alpha \in (0, 1)$ , and discount factor  $\beta \in (0, 1)$ .

**Stochastic productivity.** TFP follows an AR(1) in logs, with  $|\varrho| < 1$ :

$$\ln z_{t+1} = \varrho \ln z_t + \sigma_z \varepsilon_{t+1}, \quad \varepsilon_{t+1} \sim \mathcal{N}(0, 1).$$

**State, control, timing.** The state is  $\mathbf{x}_t = (K_t, z_t)$  and the control is  $C_t$ , with  $K_{t+1} = z_t K_t^\alpha - C_t$ . Both  $C_t$  and  $K_{t+1}$  are **chosen at date  $t$** , so both are measurable with respect to time- $t$  information. The network takes  $(K_t, \ln z_t)$  as input.

That timing assumption is what lets us pull  $1/C_t$  out of the expectation two slides from now.

# Deriving the Euler Equation (Lagrangian)

**Step 1.** Attach a multiplier  $\lambda_t$  to each period's budget:

$$\Lambda = \mathbb{E} \left[ \sum_{t=0}^{\infty} \beta^t [\ln C_t - \lambda_t (C_t + K_{t+1} - z_t K_t^\alpha)] \right].$$

**Step 2.**  $K_{t+1}$  appears **twice**, as an outflow at  $t$  and through production at  $t+1$ , so both terms enter. Dividing by  $\beta^t$ ,

$$\frac{\partial \Lambda}{\partial C_t} : \lambda_t = \frac{1}{C_t}, \quad \frac{\partial \Lambda}{\partial K_{t+1}} : \lambda_t = \beta \mathbb{E} [\lambda_{t+1} \alpha z_{t+1} K_{t+1}^{\alpha-1}].$$

**Step 3.** As  $\lambda_t$  and  $K_{t+1}$  are known at  $t$ , iterated expectations makes the expectation conditional, and  $\lambda_t = 1/C_t$  gives

$$\boxed{\frac{1}{C_t} = \beta \mathbb{E}_t \left[ \frac{\alpha z_{t+1} K_{t+1}^{\alpha-1}}{C_{t+1}} \right]}$$

Marginal utility today equals discounted expected marginal utility tomorrow, times the gross marginal product of capital at  $t+1$ .

# Closed-Form Policy via Guess-and-Verify

**Guess.** Consumption is a constant fraction of output,  $C_t = (1 - s^*) z_t K_t^\alpha$ , so the budget gives  $K_{t+1} = s^* z_t K_t^\alpha$ . Here  $s^* \in (0, 1)$  is the **savings rate**, the object the network will learn.

**Substitute and solve.** With  $C_{t+1} = (1 - s^*) z_{t+1} K_{t+1}^\alpha$  the marginal term collapses,

$$\frac{\alpha z_{t+1} K_{t+1}^{\alpha-1}}{(1 - s^*) z_{t+1} K_{t+1}^\alpha} = \frac{\alpha}{(1 - s^*) K_{t+1}}, \quad \text{so} \quad \frac{1}{(1 - s^*) z_t K_t^\alpha} = \frac{\beta \alpha}{(1 - s^*) s^* z_t K_t^\alpha},$$

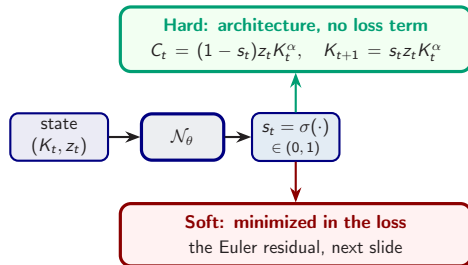
which gives  $s^* = \alpha\beta$  and therefore

$$\boxed{C_t = (1 - \alpha\beta) z_t K_t^\alpha, \quad K_{t+1} = s^* z_t K_t^\alpha, \quad s^* = \alpha\beta}$$

**Note:**  $z_{t+1}$  cancels and what survives is *measurable at t*, so  $\mathbb{E}_t[\cdot]$  passes straight through. This is special to log utility with Cobb–Douglas production and full depreciation. **Verify:** the Euler equation is necessary, not sufficient, and the candidate also satisfies transversality, since  $\beta^t \lambda_t K_{t+1} = \alpha \beta^{t+1} / (1 - \alpha\beta) \rightarrow 0$ , which rules out the explosive alternatives.

# Brock–Mirman: Hard vs. Soft Constraints

Split the conditions by what the architecture enforces exactly. The network emits a savings rate through a sigmoid head,  $s_t = \sigma(\mathcal{N}_\theta(K_t, \ln z_t))$ .



The resource constraint is free. The two hard lines add to  $C_t + K_{t+1} = z_t K_t^\alpha$ , so it holds **identically**, for any  $\theta$ , at every state.

**Why a savings head?** A softplus head on  $C_t$  would guarantee  $C_t > 0$  but could still push  $K_{t+1} < 0$ . The sigmoid savings head removes *both* failure modes before training starts. With  $\delta < 1$ , apply the same construction to cash-on-hand.

# Brock–Mirman: The Euler Residual

Write the Euler equation as a residual that vanishes exactly at the solution.

Residual in consumption units, the diagnostic

$$e_{\text{REE}} = \frac{\hat{C}_t}{C_t} - 1, \quad \hat{C}_t \equiv (u')^{-1}(\beta \mathbb{E}_t[\cdot]) = (\beta \mathbb{E}_t[\cdot])^{-1},$$

so  $e_{\text{REE}} = 10^{-3}$  means consumption is mispriced by **one tenth of one percent**.

**Tomorrow's consumption** means running the **same network again** at each shock draw:

$$z_{t+1}(\varepsilon) = e^{\varrho \ln z_t + \sigma_z \varepsilon}, \quad s_{t+1} = \sigma(\mathcal{N}_\theta(K_{t+1}, \ln z_{t+1})), \quad C_{t+1} = (1 - s_{t+1})z_{t+1}K_{t+1}^\alpha,$$

with  $K_{t+1} = s_t z_t K_t^\alpha$ .

**Partial depreciation.** With  $\delta < 1$  the marginal product becomes  $\alpha z_{t+1} K_{t+1}^{\alpha-1} + (1 - \delta)$ , and  $s^* = \alpha\beta$  holds only at  $\delta = 1$ .

# Brock–Mirman: From Residual to Training Loss

**Approximate the expectation first, then square.** With  $Q$  quadrature nodes  $\{\varepsilon_q\}$  and weights  $\{w_q\}$ ,

$$\mathcal{R}(\mathbf{x}_i) = 1 - \beta C(\mathbf{x}_i) \sum_{q=1}^Q w_q \frac{\alpha z_{t+1}(\varepsilon_q) K_{t+1}^{\alpha-1}}{C_{t+1}(\varepsilon_q)},$$

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N |\mathcal{R}(\mathbf{x}_i)|^2$$

where  $i$  indexes the  $N$  simulated states in the batch.

## Expectation inside, square outside

Replacing  $\mathbb{E}_t[\cdot]$  by a *single* simulated draw and squaring that minimizes

$$\mathbb{E}[e^2] = (\mathbb{E}[e])^2 + \text{Var}(e),$$

whose minimizer trades bias against shock variance. It is *not* the equilibrium policy.

**Why Brock–Mirman hides this.** Here  $z_{t+1}$  cancels, so the residual is zero *pathwise*, realization by realization, and both routes agree.

That cancellation is knife-edge. In any model without it, the single-draw route trains the wrong object while the loss curve still looks healthy.

# Gauss–Hermite: Exact in Low Dimensions, Costly in High

**Setup.** Shock dimension  $d_\varepsilon$ , with  $\varepsilon' \sim \mathcal{N}(0, I_{d_\varepsilon})$ : Brock–Mirman has  $d_\varepsilon = 1$ , an international real business cycle (IRBC) model with  $n_c$  countries has  $d_\varepsilon = n_c + 1$ . Gauss–Hermite uses  $Q$  nodes,

$$\mathbb{E}[h(\varepsilon)] = \int_{-\infty}^{\infty} h(\varepsilon) \frac{e^{-\varepsilon^2/2}}{\sqrt{2\pi}} d\varepsilon \approx \sum_{q=1}^Q w_q h(\varepsilon_q),$$

exact for polynomials of degree  $\leq 2Q - 1$ , spectrally convergent for analytic  $h$ .

## The rescaling that breaks student code

Library routines such as `numpy.polynomial.hermite.hermgauss` return the *physicists'* rule, for weight  $e^{-x^2}$ . Convert with  $\varepsilon_q = \sqrt{2}x_q$  and  $w_q = \tilde{w}_q/\sqrt{\pi}$ . Forgetting this gives a silently wrong expectation.

**Tensor product.** Combining one-dimensional rules costs  $Q^{d_\varepsilon}$  nodes per state: for  $Q = 3$ ,  $d_\varepsilon = 11$ , about  $1.8 \times 10^5$ .

# Stroud-3: Linear Cost in the Shock Dimension

**Idea.** Keep exactness only to total degree  $\leq 3$ , with nodes on the principal axes (Stroud 1957, 1971):

$$\mathbb{E}[h(\boldsymbol{\varepsilon}')] \approx \frac{1}{2d_\varepsilon} \sum_{k=1}^{d_\varepsilon} \left[ h(+\sqrt{d_\varepsilon} \mathbf{u}_k) + h(-\sqrt{d_\varepsilon} \mathbf{u}_k) \right]$$

so  $2d_\varepsilon$  nodes at radius  $\sqrt{d_\varepsilon}$ , with  $\mathbf{u}_k$  the  $k$ -th basis vector.

**The radius.** Matching the second moment,

$\frac{1}{2d_\varepsilon} \cdot 2r^2 = 1$ , gives  $r = \sqrt{d_\varepsilon}$ . **Exact for**  
 $\mathbb{E}[\varepsilon'_i] = \mathbb{E}[\varepsilon_i'^3] = 0$ ,  $\mathbb{E}[\varepsilon'_i \varepsilon'_j] = 0$  for  $i \neq j$ ,  
 $\mathbb{E}[\varepsilon_i'^2] = 1$ .

## Where it stops

$\mathbb{E}[\varepsilon_i'^4]$  gives  $d_\varepsilon$  against a truth of 3: the fourth-moment error **grows linearly in  $d_\varepsilon$** , exactly the regime the rule is used for. Check against a finer rule.

**Correlated shocks:** apply the rule to standard normals, then map through a Cholesky factor.

# Which Rule When

**Per-state cost**, IRBC model with  $n_c$  countries,  $d_\varepsilon = n_c + 1$  shocks ( $Q = 3$  for the tensor product):

| $n_c$ | shocks $d_\varepsilon$ | tensor product $3^{d_\varepsilon}$ | monomial $2d_\varepsilon$ | speed-up                   |
|-------|------------------------|------------------------------------|---------------------------|----------------------------|
| 2     | 3                      | 27                                 | 6                         | $\sim 5\times$             |
| 5     | 6                      | 729                                | 12                        | $\sim 60\times$            |
| 10    | 11                     | 177,147                            | 22                        | $\sim 8,000\times$         |
| 20    | 21                     | $\sim 10^{10}$                     | 42                        | $\sim 2 \times 10^8\times$ |

## Default rules of thumb

$d_\varepsilon \leq 4$ : Gauss–Hermite,  $Q = 5$ .  $5 \leq d_\varepsilon \lesssim 30$ : Stroud-3. Beyond that, or fat tails: quasi-Monte Carlo.

## Why this is cheap to get right

Only the quadrature **kernel** changes: same loss, same training loop. Validate by tightening the rule and checking that the Euler-error histogram does not move. Degree-3 exactness usually suffices near  $10^{-3}$  on IRBC-class problems (Pichler 2011; Maliar & Maliar 2014), but that is empirical.

# Implementation Gotchas That Cost a Day Each

## Detach the simulated states

The path is generated *using*  $\mathcal{N}_\theta$ . If the states are not detached from the graph, autodiff differentiates through the simulation itself. Treat simulated states as fixed data, as in Azinovic et al. (2022).

## Decide about $t+1$ deliberately

Whether the network at  $t+1$  carries gradient or is frozen is the target-network question from fitted value iteration. Either is defensible; silently switching between them is not.

## The rest of the checklist

- ▶ **Rescale the quadrature nodes** for the probabilists' convention.
- ▶ **Standardize the inputs** ( $K_t, \ln z_t$ ) before the first layer; capital and log productivity live on very different scales.
- ▶ **Burn in** the simulated path before sampling from it, so training points come from the ergodic distribution and not from  $x_0$ .
- ▶ **Evaluate the same network at every quadrature node**:  $Q$  extra forward passes per state, which is the dominant cost per step.

# Diagnostics: How Do We Know Training Worked?

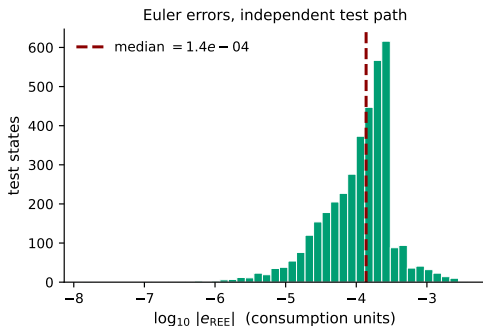
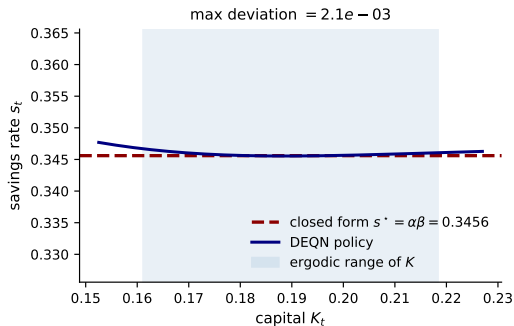
**Never trust a loss curve alone.** On an independent *test* simulation:

1. **Euler-error distribution.** Histogram  $\log_{10} |e_{\text{REE}}|$  on train and test states, in consumption units; both should peak near  $10^{-3}$  or below, with no fatter right tail on test.
2. **Policy-function benchmark.** Overlay the DEQN policy on the analytic  $s^* = \alpha\beta$ , or on a trusted reference from time iteration or perturbation.
3. **Ergodic-set coverage.** Scatter test states in  $(K_t, \ln z_t)$ : the cloud should trace the training ergodic set, neither collapsing to a point nor spreading out like a grid.
4. **Accounting and moments.** Verify budget, market clearing and non-negativity. For heterogeneous-agent models, report **den Haan (2010)** statistics.

## There is no convergence theorem

Time iteration is a contraction; DEQN training is not. The sampling distribution moves as the policy moves, so “the loss stopped falling” proves nothing. Stop on the *test* Euler-error quantiles, and report them.

# Validating a DEQN Against the Closed Form



Own computation. Stochastic Brock–Mirman,  $\alpha = 0.36$ ,  $\beta = 0.96$ ,  $\varrho = 0.8$ ,  $\sigma_z = 0.02$ , so  $s^* = 0.3456$ ; a  $2 \times 32$  tanh network with a sigmoid head, 5-node Gauss–Hermite quadrature, expectation-inside loss. **Left:** learned savings rate against the closed form. **Right:**  $\log_{10} |e_{\text{REE}}|$  on an independent test path. Note that the policy drifts *outside* the ergodic range, where it was never trained: a preview of Part VI.

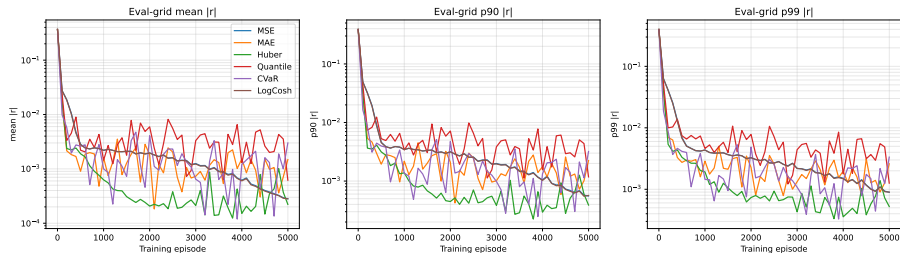
# Choice of Training Loss: Six Kernels

The loss kernel is a **modeling decision**, not a default. Write  $r = \mathcal{R}(\mathbf{x})$  for the scalar residual at a state; same residual, same network, same data, only the reduction  $r \mapsto \mathcal{L}$  changes:

| Kernel         | Formula                                          | Character                                                         |
|----------------|--------------------------------------------------|-------------------------------------------------------------------|
| MSE            | $\frac{1}{B} \sum r^2$                           | quadratic, canonical default                                      |
| MAE            | $\frac{1}{B} \sum  r $                           | constant gradient, <b>stalls at small <math> r </math></b>        |
| Huber          | quadratic for $ r  \leq \delta_H$ , linear above | robust hybrid, but <b>kink at <math> r  = \delta_H</math></b>     |
| Quantile       | pinball at $\tau = 0.9$                          | trains the upper $\tau$ -quantile only                            |
| CVaR           | mean of the top 10% of $ r $                     | explicitly trains the worst-case states                           |
| <b>LogCosh</b> | $\frac{1}{B} \sum \log \cosh(r)$                 | $C^\infty$ : $\frac{1}{2}r^2$ near 0, $ r  - \log 2$ in the tails |

Same Brock–Mirman model with full depreciation, so  $s^* = \alpha\beta$  is in closed form; same network ( $2 \times 32$  Swish, sigmoid head); same Adam optimizer; same training stream. Six runs differ *only* in the reduction applied to the residual vector.

# Loss Kernels: Convergence on Brock–Mirman



Own computation. Mean,  $p_{90}$  and  $p_{99}$  of the absolute relative Euler error on a fixed evaluation grid against training episode, log scale, 5,000 episodes per kernel.

- **LogCosh and MSE** reach the lowest mean residual here, **Huber and CVaR** the narrowest tail, and **MAE stalls** because its gradient magnitude does not shrink with the residual.
- This is one model with one architecture and the values are seed-sensitive. MSE stays the default; LogCosh is worth benchmarking against it; use Huber, Quantile or CVaR when the worst case matters.
- **Rank on the economic metric**, not the training loss: the two rankings need not agree.

# Huge Potential, but Not Yet Dynare

- ▶ **The promise:** grid-free, high-dimensional, kinks and all, as the benchmark confirms.
- ▶ **The catch:** not yet a turnkey solver, and seed-sensitive.

## Five pathologies of self-supervised training:

- Loss-scale imbalance:** normalize and adapt the weights. (Part IV)
- Architecture choice:** search the candidate space. (Part V)
- Lazy surrogate learning:** anchors and distillation. (Part VI)
- Learning from scratch:** freeze a guess, learn the correction. (Part VI)
- Self-confirming coverage:** audit on a coverage measure. (Part VI)

## Seed dependence

Same model, same code,  
different seed: one run  
converges and verifies, another  
stalls. Reproducibility is part of  
the method.

# Part IV

## Loss Balancing

# The Residual Loss Is Not One Loss

Realistic economic models minimize *several* residuals at once:

$$J(\theta) = \sum_{k=1}^M \omega_k \mathcal{L}_k(\theta).$$

In DEQNs the components are:

- ▶ Euler equation residuals
- ▶ Budget constraints not enforced by the architecture
- ▶ Market clearing
- ▶ KKT and complementarity conditions

The same structure appears in physics-informed neural networks, where the PDE, boundary and initial-condition residuals are separate components.

## The total is not a certificate

These components live at **vastly different scales**. With uniform weights the optimizer follows the **loudest** one and leaves quiet but economically important residuals large.

A small *total* loss can hide a component that is orders of magnitude too big.

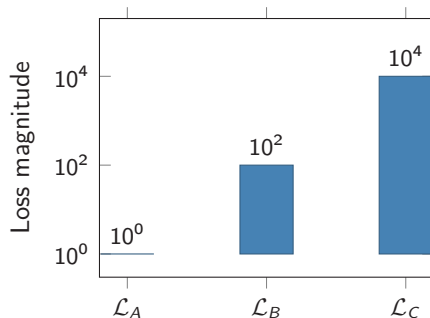
# Loss Components Differ by Orders of Magnitude

**A three-head network.** One shared trunk, one head per residual family, one loss component each:

$$J(\theta) = \omega_A \mathcal{L}_A + \omega_B \mathcal{L}_B + \omega_C \mathcal{L}_C.$$

With  $\mathcal{L}_A \approx 10^0$ ,  $\mathcal{L}_B \approx 10^2$ ,  $\mathcal{L}_C \approx 10^4$  and equal weights,  $\mathcal{L}_C$  contributes about  $10^4 \times$  more gradient signal than  $\mathcal{L}_A$ , so  $\partial J / \partial \theta \approx \omega_C \partial \mathcal{L}_C / \partial \theta$  and the network effectively solves a single-task problem.

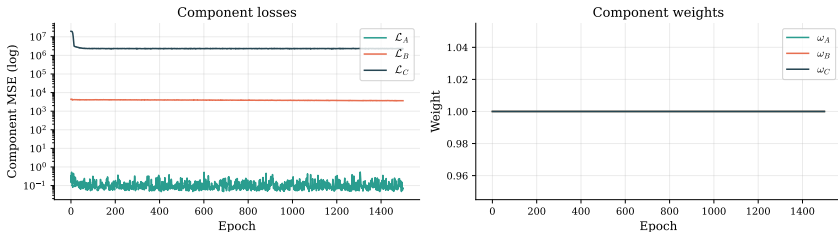
This step reads the loss ratio as a gradient ratio, which requires the heads to have comparable Jacobians. GradNorm, in the appendix, drops that assumption.



Three loss components on a log axis

# Equal Weighting: Training Dynamics

Equal weighting at scales  $10^0$ ,  $10^2$ ,  $10^4$



Own computation, notebook 02\_08, 1,500 epochs. Three Genz test functions, the standard family of tunable integrands used to benchmark approximation methods, rescaled to  $10^0$ ,  $10^2$  and  $10^4$ . Left: per-component mean squared error on a log axis; right: the constant component weights.  $\mathcal{L}_C$  drives the gradient and converges quickly,  $\mathcal{L}_B$  follows, and  $\mathcal{L}_A$  is **essentially not learned**, exactly as the scale argument predicts.

# First, Non-Dimensionalize

Every residual has a natural scale. Divide by it *before* the loss sees it:

$$\tilde{\mathcal{L}}_k = \frac{\mathcal{L}_k}{\sigma_k^2}, \quad \sigma_k = \text{the scale of residual } k.$$

In a DEQN this is not a trick, it is bookkeeping:

- ▶ Euler errors in **consumption units**, as  $e_{\text{REE}}$  in Part III
- ▶ Budget residuals as a **share of output**
- ▶ Market clearing relative to the **traded quantity**
- ▶ Complementarity residuals relative to the **multiplier's own scale**

No new hyperparameter, no schedule, no interaction with the optimizer.

## On the testbed of the next frames

Three residuals at  $10^0$ ,  $10^2$ ,  $10^4$ , five seeds, everything else held fixed. Median total relative error:

0.88

equal

→

0.008

non-dimensionalized

a factor of about **100×**, from a change of units.

Adaptive weighting is what you reach for *after* this step, for the imbalance that remains.

# Inverse-Loss Weighting

**Idea:** weight each component in inverse proportion to its current magnitude, with exponential averaging for stability:

$$\bar{\mathcal{L}}_k^{(m)} = \beta_{\text{ema}} \bar{\mathcal{L}}_k^{(m-1)} + (1 - \beta_{\text{ema}}) \mathcal{L}_k^{(m)},$$
$$\omega_k^{(m)} = \frac{1}{\bar{\mathcal{L}}_k^{(m)} + 10^{-8}},$$

then normalize so that  $\sum_k \omega_k = M$ . Typical choice:  $\beta_{\text{ema}} = 0.99$ .

**Hand-tuned weights** are the obvious alternative: they work only if the scales are known in advance, add  $M$  hyperparameters, and go stale as the losses evolve.

**Pros:** simple, no hyperparameters, adapts to changing scales.

**Cons:** unstable as  $\mathcal{L}_k \rightarrow 0$ , weights can oscillate, no notion of *relative progress*.

## The normalization bites

Forcing  $\sum_k \omega_k = M$  across losses spanning  $10^8$  drives every weight but the smallest component's to **numerical zero**. On the testbed the rule ends at  $\omega \approx (3, 0, 0)$ : it solves the quiet residual and abandons the rest.

# SoftAdapt: Weighting by Relative Progress

**Idea** (Heydari et al., 2019): weight by *relative loss changes*, so components improving slowly get more weight. With  $\Delta_k^{(m)} = \mathcal{L}_k^{(m)} - \mathcal{L}_k^{(m-1)}$ , rescaled to be dimensionless,

$$\tilde{\Delta}_k^{(m)} = \frac{\Delta_k^{(m)}}{\mathcal{L}_k^{(m)} + 10^{-8}},$$

$$\hat{\omega}_k^{(m)} = M \cdot \frac{\exp(\tilde{\Delta}_k^{(m)} / T)}{\sum_j \exp(\tilde{\Delta}_j^{(m)} / T)}.$$

- ▶  $\Delta_k > 0$ : the loss *increased*, so it needs more weight
- ▶  $\Delta_k < 0$ : it is already learning well
- ▶  $T$  is the softmax temperature: small gives winner-take-all, large gives near-equal weights

## Rescale before the softmax

The raw  $\Delta_k$  is dimensional: a component with loss near  $10^3$  produces fluctuations that swamp one near  $10^{-3}$ , so the softmax would react to units rather than to progress. The division above is not optional.

**Cons:** sensitive to  $T$ ; noisy loss changes give noisy weights.

# ReLoBRaLo: Relative Loss Balancing with Random Lookback

Three innovations (Bischof & Kraus, 2021):

1. **Relative balancing:** use *ratios*  $\mathcal{L}_k^{(m)} / \mathcal{L}_k^{(m-1)}$  instead of absolute values
2. **Baseline comparison:** mix in the **initial losses**  $\mathcal{L}_k^{(0)}$  as a stable reference
3. **Exponential smoothing:** blend new weights with history

## Why a lookback?

It prevents weight drift, resets the weights to a global perspective, and stabilizes training.

## Classroom version

In the original, the baseline mix  $\rho_{lb}$  is a *Bernoulli* draw: that randomness is the “random lookback” and the source of the stochastic regularization. We present the **deterministic** convex blend, which is what the exercises implement.

# ReLoBRaLo: The Weight Update

## Weight update at optimizer step $m$

Compute the component losses  $\mathcal{L}_1^{(m)}, \dots, \mathcal{L}_M^{(m)}$

**Step-wise ratios:**  $\kappa_{k,s}^{(m)} = \mathcal{L}_k^{(m)} / (\mathcal{T} \mathcal{L}_k^{(m-1)} + 10^{-8})$

**Baseline ratios:**  $\kappa_{k,b}^{(m)} = \mathcal{L}_k^{(m)} / (\mathcal{T} \mathcal{L}_k^{(0)} + 10^{-8})$

$\hat{\omega}_{k,s}^{(m)} = M \text{softmax}_k(\kappa_{\cdot,s}^{(m)})$ ,  $\hat{\omega}_{k,b}^{(m)} = M \text{softmax}_k(\kappa_{\cdot,b}^{(m)})$

**Combine with smoothing:**  $\omega_k^{(m)} = \alpha_{1b} [\rho_{1b} \omega_k^{(m-1)} + (1 - \rho_{1b}) \hat{\omega}_{k,b}^{(m)}] + (1 - \alpha_{1b}) \hat{\omega}_{k,s}^{(m)}$

Which way does the weight move? A *larger* ratio  $\mathcal{L}_k^{(m)} / \mathcal{L}_k^{(m-1)}$  means *less* progress this step, so the softmax gives that component *more* weight. The baseline term does the same against the starting point.

## What the two terms buy

The step-wise ratio reacts fast but is noisy; the baseline ratio is slow but anchored. The convex blend is the compromise, and  $\alpha_{1b}$  decides how much of it is memory.

# How Far Can These Weights Actually Move?

The softmax fixes the answer before any experiment. For any two components,  $\hat{\omega}_j / \hat{\omega}_k = \exp((\kappa_j - \kappa_k) / T)$ .

The step-wise  $\kappa_{k,s} = \mathcal{L}_k^{(m)} / \mathcal{L}_k^{(m-1)}$  compares **consecutive** losses, so it sits near 1 for every component *whatever its scale*; the baseline  $\kappa_{k,b}$  lies in  $[0, 1]$ .

At  $T = 1$  the achievable spread is therefore about  $e \approx 2.7$ . Measured on the testbed of the next frames:  $\omega \rightarrow (0.97, 1.20, 0.83)$ , a spread of **1.4×**.

## The consequence

A factor of 2.7 cannot close a gap of  $10^4$ . ReLoBRaLo is not failing; it answers a different question. It balances **rates of progress** and is blind to **magnitude** by construction.

**Use it where it bites:** after non-dimensionalization, when one already-comparable residual converges and another stalls. That is the setting of the original paper.

# ReLoBRaLo: Hyperparameters

| Symbol        | Name         | Default | Effect                                                        |
|---------------|--------------|---------|---------------------------------------------------------------|
| $T$           | Temperature  | 1.0     | Softmax sharpness: small is winner-take-all, large is uniform |
| $\alpha_{1b}$ | Smoothing    | 0.999   | Trust in historical weights: larger adapts more slowly        |
| $\rho_{1b}$   | Baseline mix | 0.999   | Weight on the initial-loss comparison                         |

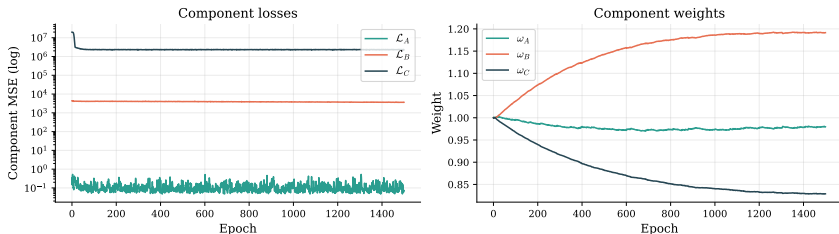
## Practical guidance

The defaults work across problems. Only  $T$  needs occasional tuning, and the sweep that follows shows why  $T \approx 1$  is a safe starting point.

The subscript “lb” is deliberate: these are *balancing* hyperparameters, and they must not be confused with the economic  $\alpha$  (capital share) and the AR(1) persistence  $\varrho$  from Part III.

# ReLoBRaLo: Weight Evolution

ReLoBRaLo at scales  $10^0$ ,  $10^2$ ,  $10^4$



Own computation, same testbed and budget as the equal-weighting figure. Left: per-component mean squared error on a log axis under ReLoBRaLo. Right: the weight trajectories  $\omega_A, \omega_B, \omega_C$ . The weights move in the direction the rule intends, and they move **barely**, against a scale gap of  $10^4$ . The loss panel is, to the eye, the equal-weighting panel.

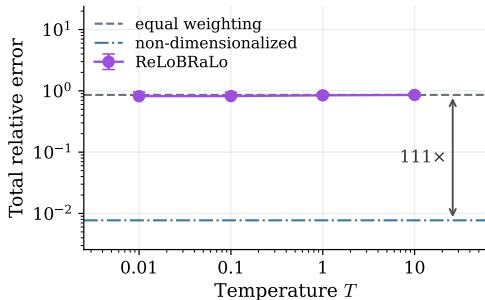
# Results: Only One of These Is a Fix

**Testbed.** Three Genz targets on  $[0, 1]^2$  at scales  $\sigma = 10^0, 10^2, 10^4$ ; shared trunk  $3 \times 64$  ReLU with three heads; Adam with  $\eta = 10^{-3}$ ; 1,000 training and 2,000 test points; 1,500 epochs. Score:  $\sum_k \text{MAE}_k / \sigma_k$ . **Median over five seeds**, range in brackets.

| Rule                                 | Total relative error | Range over seeds |
|--------------------------------------|----------------------|------------------|
| Equal weights                        | 0.86                 | [0.81, 0.92]     |
| Inverse-loss weighting               | 0.42                 | [0.38, 0.86]     |
| ReLoBRaLo, $T = 1$                   | 0.85                 | [0.81, 0.92]     |
| ReLoBRaLo, $T = 0.1$                 | 0.82                 | [0.80, 0.94]     |
| <b>Non-dimensionalized residuals</b> | <b>0.008</b>         | [0.005, 0.008]   |

- ▶ **ReLoBRaLo sits on top of doing nothing**, exactly as the softmax bound predicts.
- ▶ **Inverse-loss weighting does move**, roughly halving the total, because it reacts to *magnitude* — but it gets there by abandoning two components, and it is the least stable across seeds.
- ▶ **Rescaling the residuals is about  $110\times$  better**, and its range overlaps nothing.

# Sensitivity to the Temperature $T$



Own computation. Total relative error at the end of training, swept over  $T$ , against the two reference levels. Keep  $T \approx 1$  and spend the tuning effort on the units instead.

What  $T$  controls is the spread  $\exp(\Delta\kappa/T)$  of the previous frame: small  $T$  buys authority at the price of winner-take-all behavior, large  $T$  collapses to equal weights.

## What the sweep shows

No temperature closes the gap. All four medians sit on the equal-weighting line, two orders of magnitude above the non-dimensionalized one, and the error bars show how little separates them.

# Loss Balancing in DEQNs: What to Remember

## Takeaways:

1. Multi-component losses at different scales cause **gradient dominance**.
2. **Non-dimensionalize first**. Free, no hyperparameter, worth about  $100\times$  here: Euler errors in consumption units, budget residuals as a share of output.
3. Inverse-loss weighting is **not** a scale fix:  $\sum_k \omega_k = M$  collapses the weights onto the quietest component.
4. **ReLoBRaLo** balances *relative progress*, and its softmax can only spread the weights by about  $e^{1/T}$ . Use it *after* step 2.
5. **Monitor every component separately**, in scale-free units.

## Remedy

Rescale each residual by its own natural scale. Only then reach for adaptive weighting, and report per-component residuals rather than the total.

**References:** Bischof & Kraus (2021, arXiv:2110.09813; *CMAME* 439, 2025), ReLoBRaLo; Heydari et al. (2019), SoftAdapt; Chen et al. (2018), GradNorm; Wang, Teng & Perdikaris (2021), gradient-flow pathologies in PINNs.

# Part V

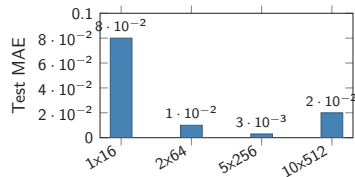
## Architecture Search

# Architecture Is Part of the Numerical Method

**Symptom.** With the *same* equations, one network converges and another stalls. Width, depth, activation and learning rate are **numerical-method choices**, not cosmetics.

## Why this matters in economics:

- ▶ **DSGE policy functions:** search improves convergence on the *same* model.
- ▶ **Surrogates:** when a network replaces a costly simulation, architecture sets approximation quality.
- ▶ **Reproducibility:** a search records which architectures were tried.



Own computation. Four architectures, same data: the test MAE moves by an order of magnitude.

## Hand-tuning does not scale

One architecture is one experiment. Reporting only the winner hides how fragile the result is.

# The Hyperparameter Space

## Architectural hyperparameters:

- ▶ Hidden layers  $L \in \{1, \dots, 5\}$
- ▶ Units per layer in  $\{32, 64, 96, \dots, 256\}$ , 8 values in steps of 32
- ▶ Activation: ReLU, tanh, Swish

## Training hyperparameters:

- ▶ Learning rate log-uniform on  $[10^{-4}, 10^{-2}]$ , on a 20-point grid
- ▶ Batch size  $B \in \{64, 128, 256\}$
- ▶ Epochs, regularization, ...

## Combinatorial explosion

Even this modest search space has

$$|\mathcal{H}| = 5 \times 8 \times 3 \times 20 \times 3 = 7,200$$

configurations, before epochs and regularization are added.

- ▶ Each trial: minutes to hours
- ▶ Full grid: **infeasible**

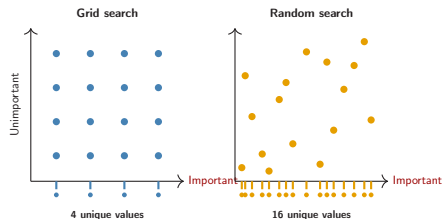
“**Grad-student descent**” (pick, train, adjust one thing, repeat until the deadline) **does not scale**, embeds human bias, and is not reproducible.

# Grid Search and Random Search

**Grid search.** Enumerate all combinations on a regular grid. Cost  $\mathcal{O}(k^{d_h})$ , exponential in the number  $d_h$  of hyperparameters. Embarrassingly parallel, but it spends most of the budget on axes that do not matter.

**Random search.** Sample configurations uniformly at random.

**The result** (Bergstra & Bengio, 2012): when only a *few* dimensions matter, random search needs *fewer* trials, because each trial contributes a fresh value on every axis.

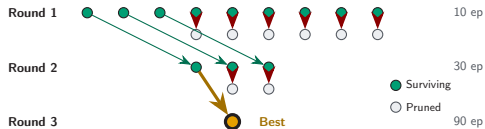


Same 16 evaluations: the grid wastes most on the unimportant axis. **The condition does the work:** with one relevant axis the median best-found value improves by  $13\times$  at  $d_h = 2$  and  $10^3\times$  at  $d_h = 8$ , but that edge **disappears** once the spare axes carry a tenth of the objective, and at  $d_h = 1$  the grid wins.

# Hyperband and Successive Halving

**Idea:** start many configurations cheaply, **stop the worst early**, give the survivors more resources. Successive halving (Karnin et al. 2013; Jamieson & Talwalkar 2016) keeps the top  $1/\gamma$  and multiplies the budget by  $\gamma$ , so *every round costs the same*.

**Hyperband** (Li et al., 2018) runs successive halving with several brackets, hedging across the unknown correlation between early and late performance. *BOHB* (Falkner et al., 2018) replaces the random sampling with Bayesian optimization.

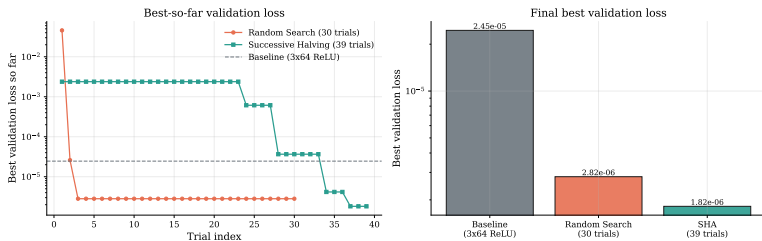


Cascade  $9 \rightarrow 3 \rightarrow 1$  at  $\gamma = 3$ . Each round costs  $9 \cdot 10 = 3 \cdot 30 = 1 \cdot 90 = 90$  epoch-equivalents, so the total is **270** against 810 for training all nine configurations fully.

# Search Results: Random Search vs. Successive Halving

**Testbed.** Two-dimensional Genz Gaussian, 1,000 training and 2,000 test points; search over layers 1–5, units 32–256, activation in {ReLU, tanh, Swish}, learning rate in  $[10^{-4}, 10^{-2}]$ ; baseline  $3 \times 64$  ReLU.

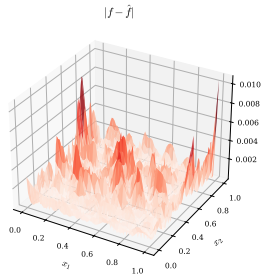
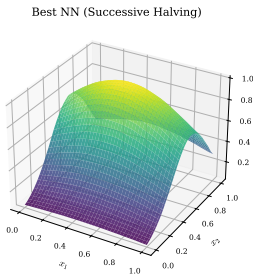
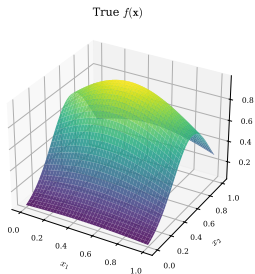
NAS on Genz Gaussian ( $d=2$ ): Random Search vs Successive Halving



Own computation. Random search uses 30 full trials; successive halving uses 27 initial candidates at  $\gamma = 3$ . Both reach a test MAE of order  $10^{-3}$  from a baseline near  $4 \cdot 10^{-3}$ , and successive halving needs about  $2.3\times$  less configuration-epoch budget in this seeded run.

# Best Model vs. Ground Truth

Best NAS model vs ground truth (MAE = 0.00099)



Own computation. Left to right: the true Genz target, the best network found by the search, and the absolute error map on  $[0, 1]^2$ . On this two-dimensional test function, and before the economic model changed at all, architecture search moved the validation loss by more than an order of magnitude.

# Architecture Search in Practice

## Search space design:

- ▶ Start small, expand if needed
- ▶ Use a **log scale** for the learning rate
- ▶ Include the current best configuration as a candidate

## Budget allocation:

- ▶ Returns diminish once the space has been explored, often after a few tens of trials
- ▶ Hyperband: the default reduction factor  $\gamma = 3$  is a reasonable start
- ▶ Fix the **compute budget up front**

## Common pitfalls:

- ▶ Search space too large, so the budget is wasted
- ▶ Overfitting the validation set, so keep a holdout test set
- ▶ Ignoring training stability, so check the learning curves too

## Remedy

Report the candidate space and the protocol, not only the winner.

# Part VI

Further Pathologies, HPC, and Wrap-Up

# Three Pathologies Left

Where we are:

- (a) Loss-scale imbalance      **done, Part IV**
- (b) Architecture choice      **done, Part V**
- (c) **Lazy surrogate learning**
- (d) **Learning from scratch**
- (e) **Self-confirming coverage**

The last three share a feature: the loss looks fine while the policy is wrong. None of them is visible on a training curve.

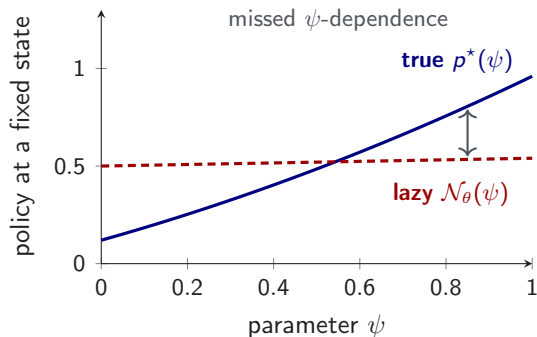
## The common thread

Residual-only training measures the policy *where the policy itself decides to look*. Every remedy in this part changes *what is measured*, not the economics.

After the pathologies: data parallelism, the hands-on preview, and the wrap-up.

# Lazy Learning: Low Residual, Wrong Policy

- **Symptom.** Lazy training is a typical pitfall of **self-supervised** (residual-only) learning: it drives the equilibrium residual down while the policy stays inaccurate across the state and parameter domain.
- Fix a state and vary a *parameter*  $\psi$ : a taste, a tax, or a shock size.
- The true policy moves with  $\psi$ ; the lazy surrogate stays *flat*, so it gets the level right on average and the **sensitivity** wrong.



At a fixed state the true policy varies with  $\psi$  (blue), while the lazy surrogate is almost flat (red dashed). The loss is low and the parameter dependence is weak: the mask is convincing only on average.

# The Fix: Anchors and Distillation

**Fix.** Augment the self-supervised residual with a small **semi-supervised distillation** term on a few trusted anchor states:

$$J^{\text{dist}}(\theta) = J(\theta) + \lambda_a \sum_{\mathbf{x}_a \in \mathcal{A}} \|\mathcal{N}_\theta(\mathbf{x}_a) - p^*(\mathbf{x}_a)\|^2.$$

- ▶  $\mathcal{A}$ : a handful of anchor states.
- ▶  $p^*$ : reference policies from an accurate solver, such as perturbation, a low-dimensional solve, or a converged reference run.
- ▶  $\lambda_a$  is one more weight, so Part IV applies to it as well.

## Remedy

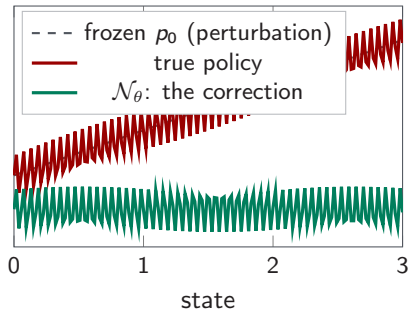
In the deep uncertainty-quantification application of Friedl et al. (2023), about **10 to 20 anchors** are typically enough to stabilize training, and leave-one-out diagnostics show the distilled surrogate agreeing with the reference policies to **within one percent**.

# Residual Learning: Freeze and Correct

**Idea.** Do not ask the network for the *whole* solution, only for the **correction** to a cheap baseline:

$$p(\mathbf{x}) = \underbrace{p_0(\mathbf{x})}_{\text{frozen guess}} + \mathcal{N}_\theta(\mathbf{x}).$$

- ▶  $p_0$ : a cheap guess (steady state, perturbation, certainty equivalent).
- ▶ The network learns only the *small, nonlinear* part, which is easier than the level and the shape at once.
- ▶ The solution is sane from iteration zero, which matters when a bad start diverges.
- ▶ → **The network is a proofreader, not an author.**



The network only fits the small wiggle: frozen guess plus learned correction.

# Residual Training Only Tests States the Policy Visits

Every DEQN solver is **self-supervised**: it drives an equation residual to zero on the states its *own* policy generates,

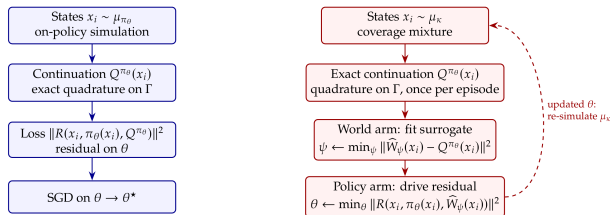
$$J(\theta) = \mathbb{E}_{\mathbf{x} \sim \mu_\theta} \|\mathcal{R}(\mathbf{x}, \mathcal{N}_\theta(\mathbf{x}))\|^2,$$

so the sampling measure  $\mu_\theta$  is generated **by the candidate policy itself**.

- ▶ The residual is only ever tested where the current policy already goes.
- ▶ A low loss says **nothing** about the states the policy avoids.
- ▶ A wrong policy visits the wrong states, so the error **hides exactly where it is never measured**.
- ▶ The loss **confirms itself**: a property of residual-only training, shared by every method in this lecture, and not a DEQN-specific flaw.

# Equilibrium World Models: A Disciplined Dreamer

Equilibrium World Models (EWM) keep the economics, changing **where** the residual is imposed.



Left: an ordinary residual solver, measuring along its own path. Right: EWM, measuring on a broader coverage measure. Schaab & Scheidegger (2026), work in progress; Dreamer lineage: Ha & Schmidhuber (2018), Hafner et al. (2023).

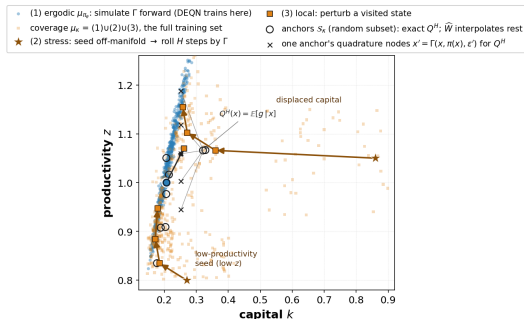
**New:** a coverage measure

$\mu_{\text{cov}} = c_1\mu_\theta + c_2\mu^{\text{stress}} + c_3\mu^{\text{local}}$  and a learned continuation value, so the residual can be evaluated off-path.

**Unchanged:** the law of motion and the residual.

The audit is stronger because it runs on  $\mu_{\text{cov}}$ , not because the economics changed.

# What the Audit Buys: Off-Path Credibility



Coverage in  $(K, z)$ : the ergodic path plus stressed and counterfactual states, each from the exact law of motion. Schaab & Scheidegger (2026).

The economics is tested in **rare, stressed and counterfactual** regions, exactly where pathwise training is weakest.

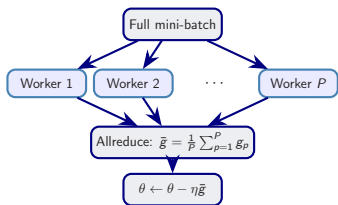
- In our experiments on **rare-disaster** models, the standard solver fails to find a verified solution across essentially every seed.
- With coverage, EWM solves in most runs and **audits** the error on the disaster region.

*Not a new equilibrium concept, but a stronger certificate for the same one.*

# Data Parallelism for DEQNs

*“To pull a bigger wagon, add more oxen rather than grow a gigantic ox.”*

a folk aphorism of parallel computing, usually attributed to Seymour Cray



**Idea:** split the mini-batch across **multiple workers**. Each computes a local gradient on its slice, and an **allreduce** averages them before the update.

- ▶ Implemented with Horovod or MPI.
- ▶ Communication costs  $\mathcal{O}(|\theta|)$  per step, **independent of the data size**: the network is exchanged, not the batch.
- ▶ Synchronous data parallelism is close to linear for moderate worker counts; large clusters become communication-limited.

# Summary: What to Take Away

## The method:

1. DEQNs replace **labels** with equations, **grids** with a network, and **time iteration** with SGD on simulated paths.
2. Enforce what you can *exactly* in the architecture; take the expectation *inside* the residual, and only then square it.
3. Validate against a benchmark; diagnose with Euler errors, never the loss curve.

## The practice:

4. **Loss-scale imbalance** is the most common silent failure: normalize and rebalance.
5. **Architecture is a numerical choice**: search it, budget it, report it.
6. A low residual can still be a **lazy**, self-confirming policy: anchor it, audit off-path.

**The one-sentence version.** Solving an economic model becomes **training a neural network**, and then all the engineering that makes that training reliable becomes part of the numerical method.

# Deep Equilibrium Nets, Part II: 11:00 – 12:30

You will build and train DEQNs yourself, starting from Brock–Mirman:

1. **Deterministic Brock–Mirman:** exogenous uniform sampling of states; compare the trained policy against the closed form  $s^* = \alpha\beta$ , and reproduce the validation figure from Part III. 02\_01
2. **Stochastic Brock–Mirman** ( $\varrho > 0$ ,  $\sigma_z > 0$ ): **simulation-based sampling** on the ergodic set, with Gauss–Hermite quadrature for  $\mathbb{E}_t[\cdot]$ , including the node rescaling. 02\_02
3. **Extensions:** endogenous labor with a non-negativity constraint, which introduces a *Karush–Kuhn–Tucker* multiplier  $\nu_t \geq 0$  and a second loss component. 02\_03, 02\_04

Then the two pathologies of Parts IV and V, on the same code: loss kernels (02\_05), architecture search (02\_06, 02\_07), and loss balancing (02\_08).

**Materials:** [course GitHub repository](#) · **Platform:** [nuvolos.cloud](#) (no local setup required)

# Exercise 3 in One Slide: KKT Without a Kink

**The problem.** Complementary slackness bundles three conditions at once,

$$\nu \geq 0, \quad g \geq 0, \quad \nu g = 0,$$

and inequalities are awkward inside a smooth loss.

**The trick.** The *Fischer–Burmeister* function (Fischer, 1992) packages all three into one residual:

$$\varphi_{\text{FB}}(a, b) = a + b - \sqrt{a^2 + b^2}, \quad \varphi_{\text{FB}}(a, b) = 0 \iff a, b \geq 0, \quad ab = 0.$$

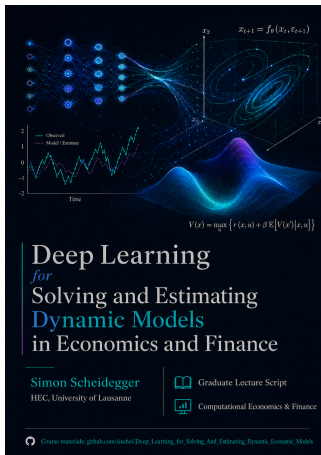
## Why the equivalence holds

For  $a, b \geq 0$ ,  $a + b = \sqrt{a^2 + b^2}$  if and only if  $(a + b)^2 = a^2 + b^2$ , that is  $2ab = 0$ . The function is differentiable everywhere except at the origin, and autodiff handles it directly.

## Where Part IV comes back

The exercise loss adds a squared *Euler* residual to a squared *Fischer–Burmeister* residual. The two live at different scales, which makes it a textbook case for the **loss balancing** of Part IV.

# Want More Details? The Full Book



## Deep Learning for Solving and Estimating Dynamic Models in Economics and Finance

A book and graduate lecture script that develops these methods in full.

- ▶ **18 lectures**, from foundations to research-level applications.
- ▶ **Everything is in code**: each lecture ships with runnable notebooks.
- ▶ Covers what this lecture could only sketch: heterogeneous agents, occasionally binding constraints, estimation, and uncertainty quantification.

Freely available: [the book repository on GitHub](#)

# Questions?



Simon Scheidegger

HEC, University of Lausanne

<https://sischei.github.io>

Next up: 11:00 – 12:30, Deep Equilibrium Nets  
(part II)

Materials:

[github.com/sischei/summer-school-AI-for-economics-and-finance-2026](https://github.com/sischei/summer-school-AI-for-economics-and-finance-2026)

Platform: [nuvolos.cloud](https://nuvolos.cloud)

Backup slides

# GradNorm: Balance the Gradients, Not the Losses

**Idea** (Chen et al., 2018): adjust  $\omega_k$  so that the *gradient norms* are balanced,

$$\|\nabla_{\mathbf{W}} \omega_k \mathcal{L}_k\|_2 \approx \bar{G} \cdot \kappa_k^{(m)},$$

by descending on the auxiliary loss

$$\mathcal{L}_{\text{grad}} = \sum_{k=1}^M \left| \|\nabla_{\mathbf{W}} \omega_k \mathcal{L}_k\|_2 - \bar{G} \cdot \kappa_k^{(m)} \right|,$$

with the target  $\bar{G} \cdot \kappa_k^{(m)}$  treated as a **constant** when differentiating. The weights are learnable,  $\omega_k \leftarrow \omega_k - \eta_{\omega} \nabla_{\omega_k} \mathcal{L}_{\text{grad}}$ , then renormalized to  $\sum_k \omega_k = M$ .

where

- ▶  $\mathbf{W}$  is a **chosen subset of shared weights**, typically the last shared layer, which is what makes the method affordable;
- ▶  $\bar{G}$  is the average gradient norm over components;
- ▶  $\kappa_k^{(m)} = (\tilde{\mathcal{L}}_k^{(m)} / \tilde{\mathcal{L}}^{(m)})^{\alpha_{\text{gn}}}$  is the relative inverse training rate, with  $\tilde{\mathcal{L}}_k^{(m)} = \mathcal{L}_k^{(m)} / \mathcal{L}_k^{(0)}$  and  $\alpha_{\text{gn}} \approx 1.5$ .

**Pros:** principled, and it addresses gradient competition directly.

**Cons:** needs per-component gradients, so it is more expensive to run and more intricate to implement.

# Bayesian Optimization

**Idea:** build a *surrogate* from configuration to loss, and use it to pick the next trial.

1. Fit a Gaussian process to the observed trials
2. Compute an **acquisition function**, such as Expected Improvement
3. Evaluate the best configuration, update, repeat

**Advantage:** **sample-efficient**: it exploits structure in the response surface.

## Expected Improvement

$$\text{EI}(\mathbf{x}) = \mathbb{E}[\max(f^* - f(\mathbf{x}), 0)]$$

with  $f^*$  the best observed value: it balances *exploration* against *exploitation*.

**Cons:** inherently sequential, so harder to parallelize; the surrogate fit costs  $\mathcal{O}(n^3)$  in the number of trials and degrades in high dimensions.

# Search Strategies Compared

| Property                  | Random         | Bayesian                  | Hyperband             |
|---------------------------|----------------|---------------------------|-----------------------|
| Trials to a given quality | Baseline       | <b>Fewest</b>             | Few                   |
| Cost per trial            | Full training  | Full training + surrogate | <b>Early stopping</b> |
| Parallelization           | Fully parallel | Sequential                | Mostly parallel       |

- ▶ Comparing raw cost in the budget  $n$  is meaningless, since  $n$  is chosen. The meaningful axis is how many trials a method needs to reach a given quality.
- ▶ Grid search is omitted: it is dominated by random search on every axis here.
- ▶ Hyperband samples configurations at random, so it handles interactions exactly as well as random search. What it adds is early stopping, not a better proposal.

Production tooling wraps all of these: Optuna, Ray Tune, KerasTuner, Ax.